



数据库系统概论

An Introduction to Database System

第三章 关系数据库标准语言SQL

刘 淇

Email: qiliuql@ustc.edu.cn

课程主页:

<http://staff.ustc.edu.cn/~qiliuql/DB2018HF.html>



四、修改基本表

2

ALTER TABLE <表名>

[ADD <新列名> <数据类型> [完整性约束]]

[DROP <完整性约束名>]

[ALTER COLUMN<列名> <数据类型>];



修改基本表（续）

3

[例8]向Student表增加“入学时间”列，其数据类型为日期型。

```
ALTER TABLE Student ADD S_entrance DATE;
```

- 不论基本表中原来是否已有数据，新增加的列一律为空值。
- 删除呢？

```
alter table Student drop S_entrance;
```

[例9]将年龄的数据类型由字符型（假设原来的数据类型是字符型）改为整数。

```
ALTER TABLE Student ALTER COLUMN Sage INT; #Mysql
```

会报错

[例10]增加课程名称必须取唯一值的约束条件。

```
ALTER TABLE Course ADD UNIQUE(Cname);
```



修改基本表（续）

4

□ MySQL（两种方法）

- alter table 表名称 change 字段名称 字段名称 字段类型 [是否允许非空];
- alter table 表名称 modify 字段名称 字段类型 [是否允许非空];

alter table Student change Sage StudentAge INT; #修改列名

alter table Student modify Sage INT not null; #mysql

□ 例如:

修改表expert_info中的字段birth,允许其为空
alter table expert_info change birth birth varchar(20) **null**;



五、删除基本表

5

DROP TABLE <表名> [RESTRICT| CASCADE] ;

- **RESTRICT:** 删除表是有限制的。
 - 欲删除的基本表不能被其他表的约束所引用
 - 如果存在依赖该表的对象，则此表不能被删除
- **CASCADE:** 删除该表没有限制。
 - 在删除基本表的同时，相关的依赖对象一起删除



删除基本表(续)

6

[例11] 删除Student表

DROP TABLE Student CASCADE;

- 基本表定义被删除，数据被删除
- 表上建立的索引、视图、触发器等一般也将被删除



删除基本表（续）

7

[例12] 若表上建有视图，选择RESTRICT时表不能删除

```
CREATE VIEW IS_Student  
AS
```

```
SELECT Sno, Sname, Sage  
FROM Student  
WHERE Sdept='IS';
```

```
DROP TABLE Student RESTRICT;
```

```
--ERROR: cannot drop table Student because other  
objects depend on it
```



删除基本表（续）

8

[例12]如果选择CASCADE时可以删除表，视图也自动被删除

```
DROP TABLE Student CASCADE;
```

```
--NOTICE: drop cascades to view IS_Student
```

```
SELECT * FROM IS_Student;
```

```
--ERROR: relation " IS_Student " does not exist
```



删除基本表（续）-不同数据库产品略有差别

9

DROP TABLE时，SQL99 与 3个RDBMS的处理策略比较

序号	标准及主流数据库的处理方式 依赖基本表的对象	SQL99		Kingbase ES		ORACLE 9i		MS SQL SERVER 2000
		R	C	R	C		C	
1.	索引	无规定		√	√	√	√	√
2.	视图	×	√	×	√	√ 保留	√ 保留	√ 保留
3.	DEFAULT, PRIMARY KEY, CHECK (只含该表的列) NOT NULL 等约束	√	√	√	√	√	√	√
4.	Foreign Key	×	√	×	√	×	√	×
5.	TRIGGER	×	√	×	√	√	√	√
6.	函数或存储过程	×	√	√ 保留	√ 保留	√ 保留	√ 保留	√ 保留

R表示RESTRICT, C表示CASCADE

'×'表示不能删除基本表, '√'表示能删除基本表, '保留'表示删除基本表后, 还保留依赖对象

注：MySQL呢？



3.3 数据定义

10

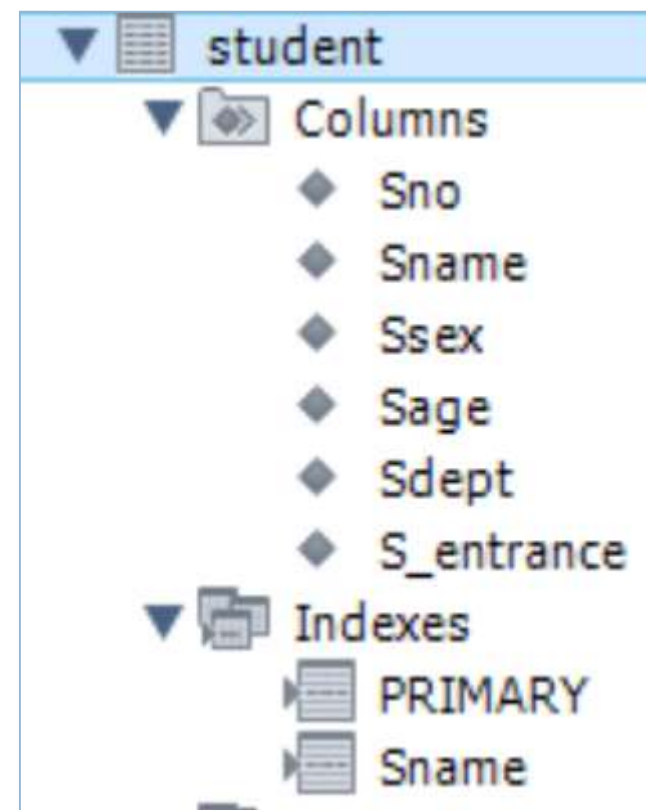
- 3.3.1 模式的定义与删除
- 3.3.2 基本表的定义、删除与修改
- 3.3.3 索引的建立与删除



3.3.3 索引的建立与删除

11

- 建立索引 (**Index**) 的目的: 加快查询速度
- 谁可以建立索引
 - DBA 或 表的属主 (即建立表的人)
 - DBMS 一般会 自动建立以下列上的索引
 - PRIMARY KEY**
 - UNIQUE**
- 谁维护索引
 - DBMS 自动完成
- 使用索引
 - DBMS 自动选择是否使用索引以及使用哪些索引





索引

12

- RDBMS中索引一般采用**B+树**、**HASH**索引来实现
 - B+树索引具有动态平衡的优点
 - HASH索引具有查找速度快的特点
- 采用B+树，还是HASH索引 则由具体的RDBMS来决定
- 索引是关系数据库的内部实现技术，属于**内模式**的范畴
- **CREATE INDEX**语句定义索引时，可以定义索引是唯一索引、非唯一索引或聚簇索引
- 聚簇索引：元组按照索引键值的顺序存储



B+/-树索引

13

□ 从二叉树讲起

二叉排序树上**查找**某关键字
等于给定值的结点过程，其
实就是走了一条从根到该结
点的路径。

比较的关键字次数

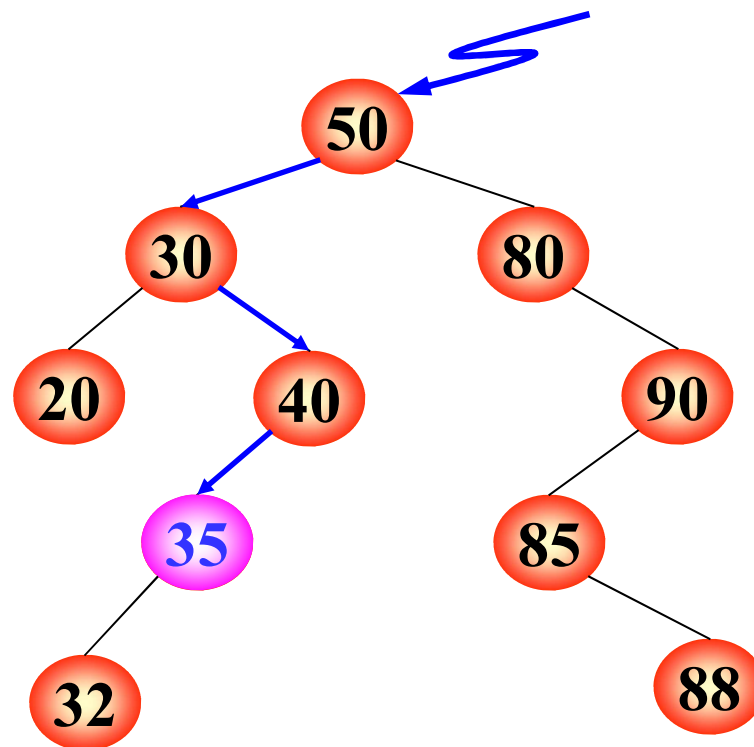
||

此结点所在层次数

最多的比较次数

||

树的深度



查找关键字：35

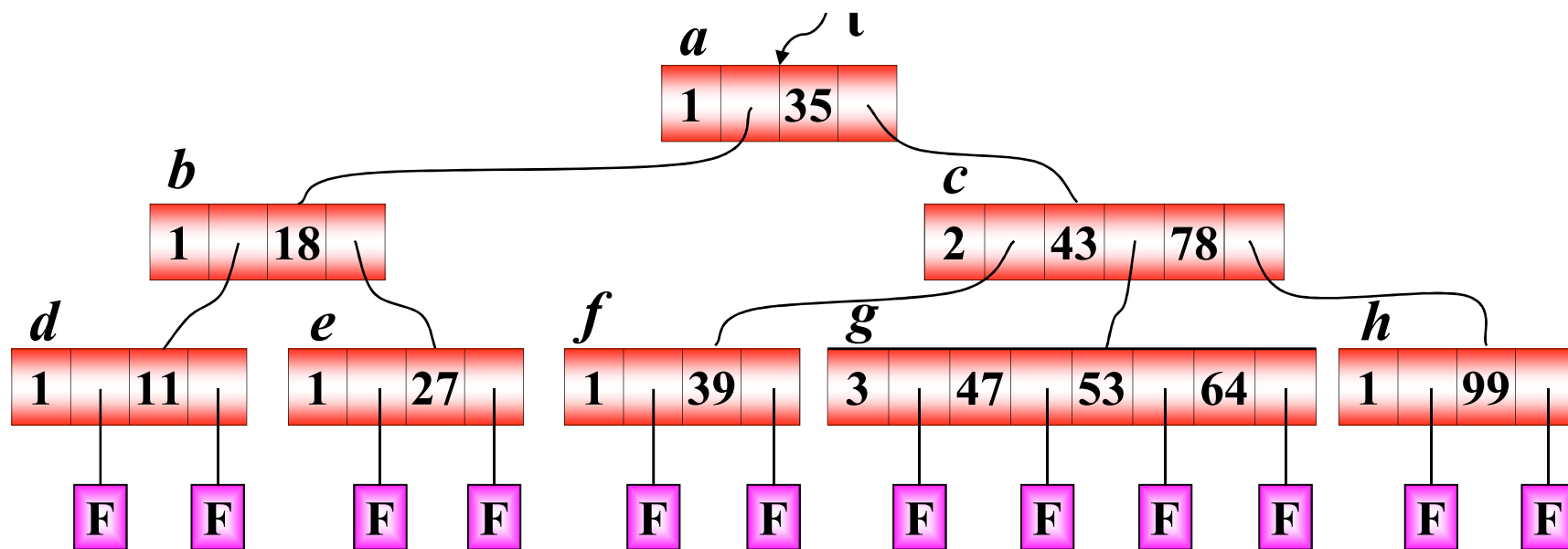


B+/-树—B-树索引

14

一棵 m 阶的 **B-** 树，或为空树或为满足下列特性的 m 叉树：

- (5)、所有叶子结点在同一个层次上，且不含有任何信息。（可以看作是外部结点或查找失败的结点，实际上这些结点不存在，指向这些结点的指针为空）。





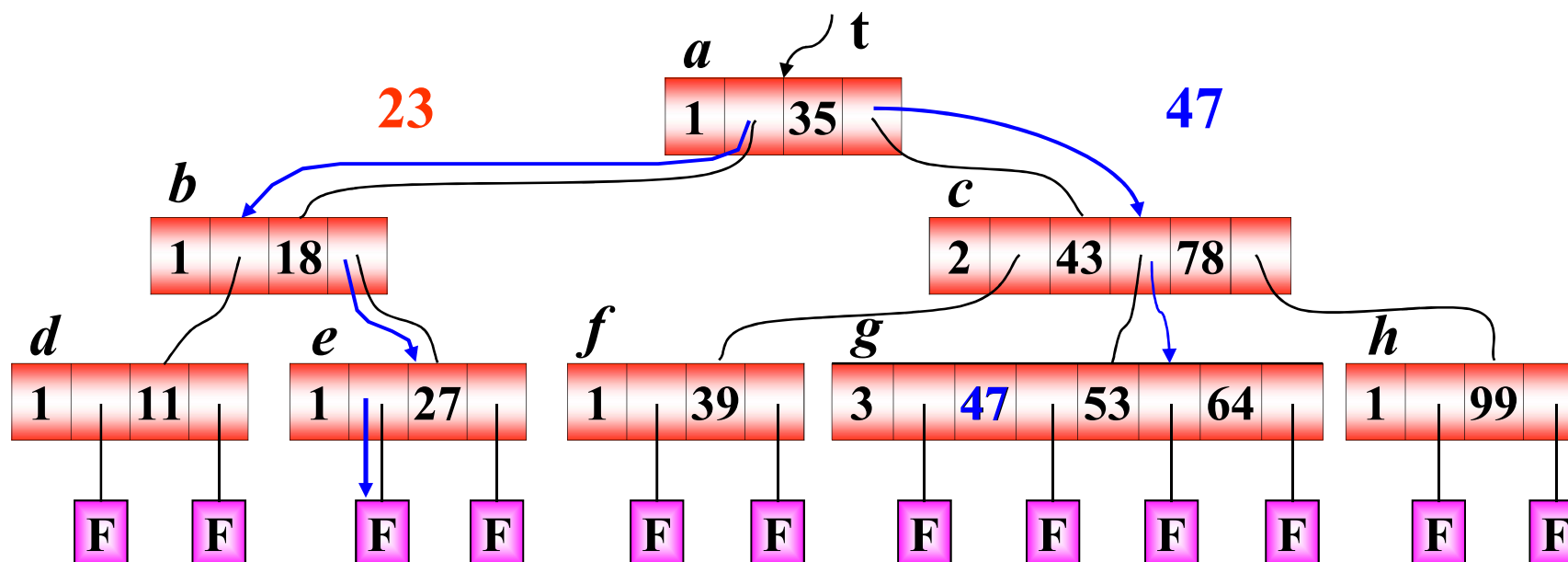
B+/-树—B-树索引

15

从根结点出发，沿指针**搜索结点**和在**结点内进行**顺序（或折半）**查找**两个过程交叉进行。

若**查找成功**，则**返回指向**被查关键字所在**结点的指针**和**关键字在结点中的位置**；

若**查找不成功**，则**返回插入位置**。





B+树索引

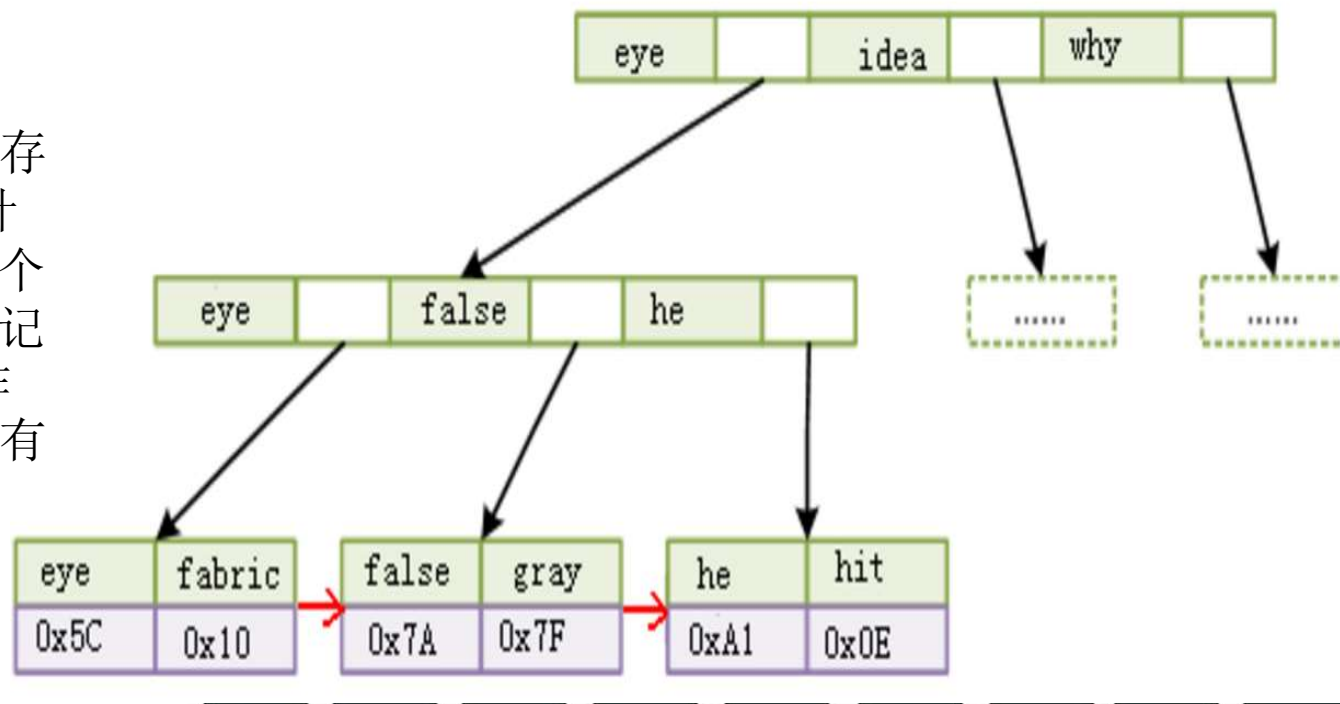
16

B+树是B-树的变体，也是一种多路搜索树：

- 1.其定义基本与B-树同，除了：
- 2.非叶子结点的子树指针与关键字个数相同；
- 3.非叶子结点的子树指针 $P[i]$ ，指向关键字值属于 $[K[i], K[i+1])$ 的子树（B-树是开区间）；
- 5.为所有叶子结点增加一个链指针；
- 6.所有关键字都在叶子结点出现；

如：（ $M=3$ ）

B+Tree: 非叶子节点只存key，大大滴减少了非叶子节点的大小，那么每个节点就可以存放更多的记录，树更矮了，I/O操作更少了。所以B+Tree拥有更好的性能。





Hash表索引

17

- 预先知道所查关键字在表中的位置。即，记录在表中的位置和其关键字之间存在一种确定的关系。

例如：为每年招收的 **1000** 名新生建立一张查找表，其关键字为学号，其值的范围为 **xx000 ~ xx999**（前两位为年份）。

若以下标为**000 ~ 999**的有序顺序表表示之，则查找过程可以简单进行：取给定值（学号）的后三位，不需要经过比较便可直接从顺序表中找到待查关键字。

上例表明，记录的**关键字**与记录在表中的**存储位置**之间存在一种对应（函数）关系。若记录的关键字为 key ，记录在表中的位置（称为**哈希地址**）为 $f(key)$ ，则称此函数 $f(key)$ 为**哈希函数**（散列函数）。



Hash表

18

例如：对于如下 9 个关键字：

{Zhao, Qian, Sun, Li, Wu, Chen, Han, Ye, Dai}

设哈希函数 $f(\text{key}) = \lfloor (\text{Ord}(\text{关键字首字母}) - \text{Ord}('A') + 1) / 2 \rfloor$

0	1	2	3	4	5	6	7	8	9	10	11	12	13
	Chen	Dai		Han		Li		Qian	Sun		Wu	Ye	Zhao

问题：若添加关键字 Zhou，会出现什么情况？

Zhou

从这个例子可见：

- 1) 哈希函数是一个映像，即：将关键字的集合映射到某个地址集合上。它的设置很灵活，只要使得关键字的哈希函数值都落在表长允许的范围之内即可；
- 2) 由于哈希函数是一个压缩映像，因此，在一般情况下，很容易产生“冲突”现象，即： $\text{key1} \neq \text{key2}$ ，而 $f(\text{key1}) = f(\text{key2})$ 。这种具有相同函数值的关键字称为同义词。



Hash表

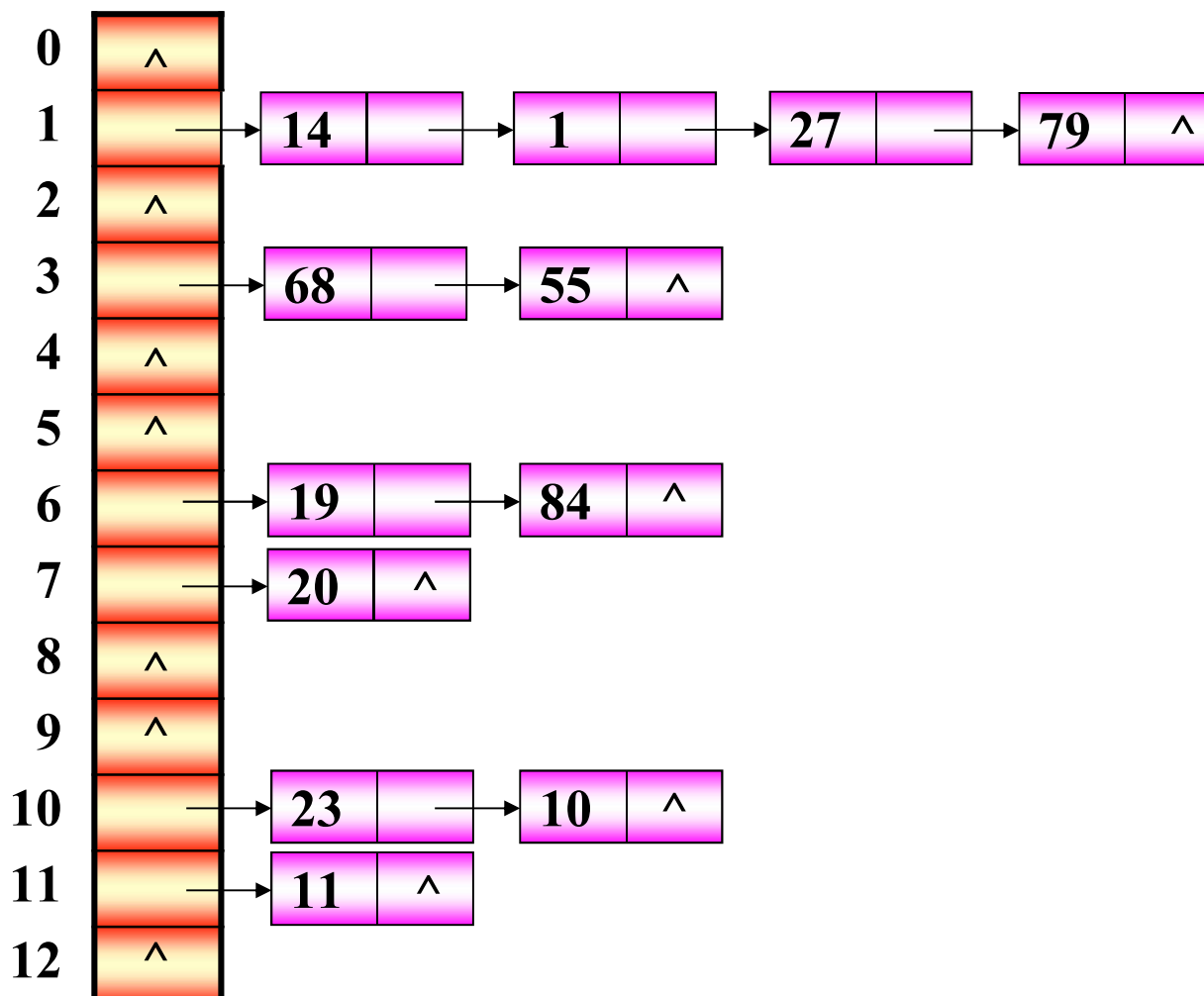
19

- “处理冲突”的实际含义是：为产生冲突的地址寻找另一个哈希地址。

例：已知一组关键字 (19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79)

哈希函数为：

$H(\text{key}) = \text{key} \bmod 13$ ，用链地址法处理冲突。





一、建立索引

20

□ 语句格式

```
CREATE [UNIQUE] [CLUSTER] INDEX <索引名>  
ON <表名>(<列名>[<次序>][,<列名>[<次序>] l...);
```

mysql中没有cluster index

聚簇索引：元组按照索引键值的顺序存储



建立索引（续）

21

- **聚簇索引**是顺序结构与数据存储物理结构一致的一种索引
 - 通常物理顺序结构只有一种，那么一个表的聚簇索引也只能有一个，设置了主码，系统默认就为表加上了聚簇索引。若需要用其他字段作为索引，可在设置主码之前自己手动的先添加上唯一的聚簇索引，然后再设置主码。
 - **B+Tree**的叶子节点上的**data**就是数据本身（主码或其它字段）
- **非聚簇索引**记录的物理顺序与逻辑顺序没有必然的联系，与数据的存储物理结构没有关系
 - 一个表对应的非聚簇索引可以有多条，根据不同列的约束可以建立不同要求的非聚簇索引
 - **B+Tree**的叶子节点上的**data**，并不是数据本身，而是数据存放的地址

细节在第7.5.2节



建立索引（续）

22

[例13]在Student表的Sname（姓名）列上建立一个聚簇索引

```
CREATE CLUSTER INDEX Stusname  
ON Student(Sname);
```

- 在最经常查询的列上建立聚簇索引以提高查询效率
- 一个基本表上最多只能建立一个聚簇索引
- 经常更新的列不宜建立聚簇索引



建立索引（续）

23

[例14]为学生-课程数据库中的Student, Course, SC三个表建立索引。

Student表按学号升序建唯一索引

Course表按课程号升序建唯一索引

SC表按学号升序和课程号降序建唯一索引

```
CREATE UNIQUE INDEX Stusno ON Student(Sno);
```

```
CREATE UNIQUE INDEX Coucno ON Course(Cno);
```

```
CREATE UNIQUE INDEX SCno ON SC(Sno ASC, Cno DESC);
```



二、删除索引

24

□ **DROP INDEX** <索引名>;

删除索引时，系统会从数据字典中删去有关该索引的描述。

[例15] 删除Student表的Stusname索引

DROP INDEX Stusname;

DROP INDEX Stusname on student;



3.5 数据更新

25

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据



3.5.1 插入数据

26

- 两种插入数据方式
 1. 插入元组
 2. 插入子查询结果
- 可以一次插入多个元组



一、插入元组

27

□ 语句格式

INSERT

INTO <表名> [(<属性列1>[, <属性列2>...])]

VALUES (<常量1> [, <常量2>] ...)

□ 功能

- 将新元组插入指定表中



插入元组（续）

28

- **INTO**子句
 - 属性列的顺序可与表定义中的顺序不一致
 - 没有指定属性列
 - 指定部分属性列
- **VALUES**子句
 - 提供的值必须与**INTO**子句匹配
 - 值的个数
 - 值的类型

**Into子句没有出现的属性列，新元组在这些列上取Null;
在表定义时说明了Not Null的属性列不能取空，否则报错；**



插入元组（续）

29

[例1] 将一个新学生元组（学号：200215128；姓名：陈春；性别：男；所在系：IS；年龄：18岁）插入到 Student表中。

```
1 • INSERT
2     INTO Student (Sno, Sname, Ssex, Sdept, Sage)
3     VALUES ("200215129", "陈春", "男", "IS", 18);
4 • SELECT * FROM student:
```

result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
Sno	Sname	Ssex	Sage	Sdept
200215128	陈冬	男	18	IS
200215129	陈春	男	18	IS

注意单引号或者双引号的使用



插入元组（续）

30

[例2] 将学生张成民的信息插入到Student表中。

INSERT

INTO **Student**

VALUES ('200215126', '张成民', '男', 18, 'CS');

```
1 • INSERT
2 INTO Student
3 VALUES ("200215126", "张成民", "男", 18, "CS");
4 • SELECT * FROM student;
```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
Sno	Sname	Ssex	Sage	Sdept
200215126	张成民	男	18	CS
200215128	陈冬	男	18	IS



插入元组（续）

31

[例3] 插入一条选课记录('200215128', '1 ')。

INSERT

INTO SC(Sno, Cno)

VALUES (' 200215128 ', ' 1 ');

RDBMS将在新插入记录的**Grade**列上自动地赋空值。

或者：

INSERT

INTO SC

VALUES (' 200215128 ', ' 1 ', NULL);



查询表中有哪些属性列

32

- SHOW COLUMNS FROM 表名;

2 • SHOW COLUMNS FROM student;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
	Sno	char(9)	NO	PRI	NULL	
▶	Sname	char(20)	YES	UNI	NULL	
	Ssex	char(2)	YES		NULL	
	Sage	int(11)	NO		NULL	
	Sdept	char(20)	YES		NULL	



二、插入子查询结果 (select学过以后再介绍)

33

□ 语句格式

INSERT

INTO <表名> [(<属性列1> [, <属性列2>...])

子查询;

□ 功能

将子查询结果插入指定表中



插入子查询结果（续）

34

- INTO子句(与插入元组类似)
- 子查询
 - SELECT子句目标列必须与INTO子句匹配
 - 值的个数
 - 值的类型



插入子查询结果（续）

35

[例4] 对每一个系，求学生的平均年龄，并把结果存入数据库。

第一步：建表

```
CREATE TABLE Dept_age  
  (Sdept CHAR(15)          /* 系名*/  
   Avg_age SMALLINT);     /*学生平均年龄*/
```



插入子查询结果（续）

36

第二步：插入数据

```
INSERT  
INTO Dept_age(Sdept, Avg_age)  
SELECT Sdept, AVG(Sage)  
FROM Student  
GROUP BY Sdept;
```



插入子查询结果（续）

37

RDBMS在执行插入语句时会检查所插元组是否破坏表上已定义的完整性规则

- 实体完整性
- 参照完整性
- 用户定义的完整性
 - **NOT NULL**约束
 - **UNIQUE**约束
 - 值域约束



3.5 数据更新

38

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据



3.4.2 修改数据

39

□ 语句格式

UPDATE <表名>

SET <列名>=<表达式>[, <列名>=<表达式>]...

[WHERE <条件>];

□ 功能

- 修改指定表中满足**WHERE**子句条件的元组



修改数据（续）

40

■ SET子句

- 指定修改方式
- 要修改的列
- 修改后取值

■ WHERE子句

- 指定要修改的元组
- 缺省表示要修改表中的所有元组



修改数据（续）

41

□ 三种修改方式

1. 修改某一个元组的值
2. 修改多个元组的值
3. 带子查询的修改语句



1. 修改某一个元组的值

42

[例5] 将学生200215121的年龄改为22岁

```
1 • UPDATE Student
2   SET Sage=22
3   WHERE Sno="200215126";
4 • SELECT * FROM student;
```

ult Grid Filter Rows: Edit: Export/Import:				
Sno	Sname	Ssex	Sage	Sdept
200215126	张成民	男	22	CS
200215128	陈冬	男	18	IS
200215129	陈春	男	18	IS



2. 修改多个元组的值

43

[例6] 将某个学生的年龄增加1岁

1 •
2
3
4 •

```
UPDATE Student  
SET Sage= Sage+1  
WHERE Sno="200215126";  
SELECT * FROM student;
```

ult Grid | Filter Rows: | Edit: | Export/Import: | Wr

Sno	Sname	Ssex	Sage	Sdept
200215126	张成民	男	23	CS



3. 带子查询的修改语句

44

[例7] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC  
SET Grade=0  
WHERE 'CS'=  
    (SELETE Sdept  
    FROM Student  
    WHERE Student.Sno = SC.Sno);
```



修改数据（续）

45

RDBMS在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则

- 实体完整性
- 主码不允许修改
- 用户定义的完整性
 - **NOT NULL**约束
 - **UNIQUE**约束
 - 值域约束



3.5 数据更新

46

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据



3.5.3 删除数据

47

- 语句格式

DELETE

FROM <表名>

[WHERE <条件>];

- 功能

- 删除指定表中满足**WHERE**子句条件的元组

- **WHERE**子句

- 指定要删除的元组
- 缺省表示要删除表中的全部元组，表的定义仍在字典中



删除数据（续）

48

- 三种删除方式
 - 1. 删除某一个元组的值
 - 2. 删除多个元组的值
 - 3. 带子查询的删除语句



1. 删除某一个元组的值

49

[例8] 删除学号为200215128的学生记录。

DELETE

FROM Student

WHERE Sno= '200215128 ';

```
1 • DELETE
2 FROM Student
3 WHERE Sno="200215128";
4 • SELECT * FROM student:
```

result Grid		 Filter Rows:	Edit: 	 	Export/Import: 
Sno	Sname	Ssex	Sage	Sdept	
200215126	张成民	男	23	CS	
200215129	陈春	男	18	IS	



2. 删除多个元组的值

50

[例9] 删除所有的学生选课记录。

DELETE

FROM SC;



3. 带子查询的删除语句

51

[例10] 删除计算机科学系所有学生的选课记录。

```
DELETE  
FROM SC  
WHERE 'CS' =  
    (SELETE Sdept  
        FROM Student  
        WHERE Student.Sno=SC.Sno);
```



练习

52

- SQL语句中修改表结构的命令是（ ）
A、ALTER TABLE B、MODIFY TABLE
C、ALTER STRUCTURE D、MODIFY STRUCTURE
- 删除表的SQL命令是（ ），删除数据的命令是----？
A、DROP TABLE B、DELETE TABLE
C、ERASE TABLE D、KILL TABLE
- 在右Student表中输入
 - INSERT INTO Student
 - VALUES ("200215128", "张三", "男", 18, "CS");结果系统报错，可能的原因是什么？

Sno	Sname	Ssex	Sage	Sdept
200215126	张成民	男	22	CS
200215128	陈冬	男	18	IS
200215129	陈春	男	18	IS

已知一组主码分别为 (19, 14, 23, 21) 哈希函数是： $H(\text{key}) = \text{key} \bmod 7$ ，它们在哈希索引中的地址分别是？



练习

53

- 下列SQL语句中，实现数据检索的语句是（ ），修改表结构的是（ ），修改属性值的是（ ），删除表结构的是（ ），删除表记录的是（ ）
 - A)SELECT B)DROP C)UPDATE D)ALTER E)DELETE
- 以下哪个不是SQL语句的注释方法（ ）
 - A. /* */ B. # C. -- D. //
- 建立索引的目的是（ ），自动建立索引的属性列是（ ）
 - A.加速建表 B.加快存取 C.提高安全性 D.节省空间
 - E.Primary Key F. Not Null G. Unique H. Foreign Key



练习

54

有两个关系 $R(A,B,C,D)$ 和 $S(B,C,E)$,将 R 和 S 按属性 B 进行等值连接, 到的结果包含 () 个列, 自然连接 () 。

- ☐ A.4 B.5 C.6 D.7

关系代数中, 一般 (等值) 连接操作由 () 组合而成, 自然连接 () 。

- ☐ A.笛卡尔积、选择和投影 B.笛卡尔积、选择
☐ C.笛卡尔积、投影 D.选择、投影

设关系 R 、 S 、 W 各有10个元组, 则它们笛卡尔积的基数为 () , 笛卡尔积的度 (目) 是 () 。如果它们各有10个属性且没有公共属性, 则自然连接以后的度是 () (自然连接转化为笛卡尔积), 如果有公共属性 () 。

- ☐ A.30 B. 1000 C. 10 D.小于或等于30 E.无法确定

一般而言, 若关系 R 有10个属性、 S 有5个属性, 其中各有4个公共属性, 则 $R \div S$ 还剩 () 个属性, $S \div R$ 呢 ()



练习

55

关系 R、S 如下表所示，

$R \div (\pi_{A1, A2}(\sigma_{1 < 3}(S)))$ 的结果为--

R 关系			S 关系		
A1	A2	A3	A1	A2	A4
a	b	c	a	z	a
b	a	d	b	a	h
c	d	d	c	d	d
d	f	g	d	s	c



练习

56

首先看 $\pi_{A1, A2}(\sigma_{1 < 3}(S))$ 得到哪些元组的一个关系表。

(1) $\sigma_{1 < 3}(S)$ 的意思就是从关系 S 中选择第 1 列小于第 3 列的元组组成的关系表，如表 1 所示。

表1

S 关系		
A1	A2	A4
b	a	h
c	d	d



练习

57

(2) $\pi_{A1, A2}$ 的意思就是对表 1 进行投影, 对 A1 和 A2 列投影出来, 如表 2 所示。

表2

S 关系	
A1	A2
b	a
c	d



练习

58

(3) 表2是具有两个属性A1、A2和两个元组的关系表。

接下来是 $R \div X2$ 了，首先，在R中找到A1与A2列和表2完全一致的元组。

表3

R 关系		
A1	A2	A3
a	b	c
b	a	d
c	d	d
d	f	g

然后，在表3中，说明R关系内存在A1、A2列元组与表2的所有元组相同，此时关键是看R关系中其他列在这两行元组的值是否相同。只有相同时，除法的结果就为这个值，不相同，则除法的结果为空。

所以： $R \div X2 = \{d\}$



练习

59

例题2(数据库系统工程师2005年5月试题44)

设有如下关系：关系代数表达式 $R \div S$ 的运算结果是__

关系R				关系S		
A	B	C	D	C	D	E
2	1	a	c	a	c	5
2	2	a	d	a	c	2
3	2	b	d	b	d	6
3	2	b	c			
2	1	b	d			

A.

A	B
2	1
3	2

B.

A	B
2	1

C.

A	B
a	c
b	d

D.

A	B	E
2	1	5
2	1	2



建表--练习

60

□ 设有如下实体：

学生：学号、单位、姓名、性别、年龄； 课程：编号、课程名、开课单位；

教师：教师号、姓名、性别、职称、单位； 单位：单位名称、电话

□ 上述实体中存在如下联系：

(1) 一个学生可选修多门课程，一门课程可为多个学生选修；

(2) 一个教师可讲授多门课程，一门课程可为多个教师讲授；

(3) 一个单位可有多个教师，一个教师只能属于一个单位。

□ 根据你之前创建的基本表，写出以下操作的SQL语句：

(1) 向学生表增加“出生时间”列，其数据类型为日期型

(2) 在单位表中加入“负责教师号”列，其数据必须是教师表中已有的教师。然后，删除单位表中的“电话”一列

(3) 按照学生的年龄升序为学生表建立索引，并为课程表增加课程名必须取唯一值的约束条件



第三章 关系数据库标准语言SQL

61

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 数据查询

3.5 数据更新

3.6 视图

3.7 小结



数据查询

62

□ 语句格式

```
SELECT [ALL|DISTINCT] <目标列表达式>  
[ , <目标列表达式> ] ...  
  
FROM <表名或视图名> [ , <表名或视图名> ] ...  
  
[ WHERE <条件表达式> ]  
  
[ GROUP BY <列名1> [ HAVING <条件表达式> ] ]  
  
[ ORDER BY <列名2> [ ASC|DESC ] ];
```



3.4 数据查询

63

- 3.4.1 单表查询
- 3.4.2 连接查询
- 3.4.3 嵌套查询
- 3.4.4 集合查询
- 3.4.5 Select语句的一般形式



3.4.1 单表查询

64

- 查询仅涉及一个表：
 - 一、 选择表中的若干列
 - 二、 选择表中的若干元组
 - 三、 **ORDER BY**子句
 - 四、 聚集函数
 - 五、 **GROUP BY**子句



一、选择表中的若干列

65

□ 查询指定列

[例1] 查询全体学生的学号与姓名。

```
SELECT Sno, Sname  
FROM Student;
```

The screenshot shows a database query window. At the top, the SQL query is entered: `1 • SELECT Sno, Sname` and `2 FROM Student;`. Below the query, there is a 'Result Grid' section. It contains a table with two columns: 'Sno' and 'Sname'. The table has two data rows: one with '200215126' and '张成民', and another with '200215129' and '陈春'. The interface also includes a 'Filter Rows' field and an 'Edit' button.

Sno	Sname
200215126	张成民
200215129	陈春

[例2] 查询全体学生的姓名、学号、所在系。

```
SELECT Sname, Sno, Sdept  
FROM Student;
```

类似于关系代数中的哪一个操作？



2. 查询全部列

66

- 选出所有属性列：
 - 在**SELECT**关键字后面列出所有列名
 - 将<目标列表表达式>指定为 *

[例3] 查询全体学生的详细记录。

```
SELECT Sno, Sname, Ssex, Sage, Sdept  
FROM Student;
```

或

```
SELECT *  
FROM Student;
```



3. 查询经过计算的值

67

- **SELECT**子句的<目标列表表达式>可以为:
 - 算术表达式
 - 字符串常量
 - 函数
 - 列别名



查询经过计算的值（续）

68

[例4] 查全体学生的姓名及其出生年份。

```
SELECT Sname, 2004-Sage /*假定当年的年份为2004年*/  
FROM Student;
```

输出结果:

Sname	2004-Sage
-------	-----------

李勇	1984
刘晨	1985
王敏	1986
张立	1985



查询经过计算的值（续）

69

[例5] 查询全体学生的姓名、出生年份和所有系，要求用小写字母表示所有系名

```
SELECT Sname, 'Year of Birth:', 2004-Sage,  
        LOWER(Sdept)  
FROM Student;
```

输出结果:

Sname 'Year of Birth:' 2004-Sage LOWER(Sdept)

李勇	Year of Birth:	1984	cs
刘晨	Year of Birth:	1985	is
王敏	Year of Birth:	1986	ma
张立	Year of Birth:	1985	is



查询经过计算的值（续）

70

- 使用列别名改变查询结果的列标题:

```
SELECT Sname NAME, 'Year of Birth: ' BIRTH,  
       2000-Sage BIRTHDAY, LOWER(Sdept) DEPARTMENT  
FROM Student;
```

输出结果:

NAME	BIRTH	BIRTHDAY	DEPARTMENT
李勇	Year of Birth:	1984	cs
刘晨	Year of Birth:	1985	is
王敏	Year of Birth:	1986	ma
张立	Year of Birth:	1985	is



3.4.1 单表查询

71

- 查询仅涉及一个表：
 - 一、 选择表中的若干列
 - 二、 选择表中的若干元组
 - 三、 **ORDER BY**子句
 - 四、 聚集函数
 - 五、 **GROUP BY**子句



二、选择表中的若干元组

72

□ 1. 消除取值重复的行

如果没有指定**DISTINCT**关键词，则缺省为**ALL**

[例6] 查询选修了课程的学生学号。

```
SELECT Sno FROM SC;
```

等价于：

```
SELECT ALL Sno FROM SC;
```

执行上面的**SELECT**语句后，结果为：

Sno

200215121

200215121

200215121

200215122

200215122



消除取值重复的行（续）

73

- 指定**DISTINCT**关键词，去掉表中重复的行

```
SELECT DISTINCT Sno  
FROM SC;
```

执行结果：

<u>Sno</u>
200215121
200215122



2. 查询满足条件的元组

表3.4 常用的查询条件

查询条件	谓 词
比 较	=, >, <, >=, <=, !=, <>, !>, !<; NOT +上述比较运算符
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空 值	IS NULL, IS NOT NULL
多重条件（逻辑运算）	AND, OR, NOT



(1) 比较大小

75

[例7] 查询计算机科学系全体学生的名单。

```
SELECT Sname  
FROM Student  
WHERE Sdept='CS';    # WHERE Sdept="CS";
```

[例8] 查询所有年龄在20岁以下的学生姓名及其年龄。

```
SELECT Sname, Sage  
FROM Student  
WHERE Sage < 20;
```

[例9] 查询考试成绩有不及格的学生的学号。

```
SELECT DISTINCT Sno  
FROM SC  
WHERE Grade < 60;
```



(2) 确定范围

76

□ 谓词: BETWEEN ... AND ...

NOT BETWEEN ... AND ...

[例10] 查询年龄在20~23岁（包括20岁和23岁）之间的学生的
姓名、系别和年龄

```
SELECT Sname, Sdept, Sage  
FROM Student  
WHERE Sage BETWEEN 20 AND 23;
```

[例11] 查询年龄不在20~23岁之间的学生姓名、系别和年龄

```
SELECT Sname, Sdept, Sage  
FROM Student  
WHERE Sage NOT BETWEEN 20 AND 23;
```



(3) 确定集合

77

□ 谓词：IN <值表>, NOT IN <值表> <值表>是一个集合

[例12]查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别。

```
SELECT Sname, Ssex
FROM Student
WHERE Sdept IN ( 'IS', 'MA', 'CS' ); #("IS", "MA", "CS");
```

[例13]查询既不是信息系、数学系，也不是计算机科学系的学生的姓名和性别。

```
SELECT Sname, Ssex
FROM Student
WHERE Sdept NOT IN ( 'IS', 'MA', 'CS' );
```



(4) 字符匹配

78

□ 谓词: [NOT] LIKE '<匹配串>' [ESCAPE '<换码字符>']

1) 匹配串为固定字符串

[例14] 查询学号为200215121的学生的详细情况。

```
SELECT *  
FROM Student  
WHERE Sno LIKE '200215121';
```

等价于:

```
SELECT *  
FROM Student  
WHERE Sno = '200215121';
```



字符匹配（续）

79

2) 匹配串为含通配符的字符串

%: 任意多个字符; _: 单个字符（汉字为2个字符）

【例15】 查询所有姓刘学生的姓名、学号和性别。

```
SELECT Sname, Sno, Ssex  
FROM Student  
WHERE Sname LIKE '刘%';
```

【例16】 查询姓"欧阳"且全名为三个汉字的学生的姓名。

```
SELECT Sname  
FROM Student  
WHERE Sname LIKE '欧阳__';
```



字符匹配（续）

80

[例17] 查询名字中第2个字为"阳"字的学生的姓名和学号。

```
SELECT Sname, Sno  
FROM Student  
WHERE Sname LIKE '__阳%';
```

[例18] 查询所有不姓刘的学生姓名。

```
SELECT Sname, Sno, Ssex  
FROM Student  
WHERE Sname NOT LIKE '刘%';
```



字符匹配（续）

81

3) 使用换码字符 **ESCAPE** **** 将通配符转义为普通字符

[例19] 查询DB_Design课程的课程号和学分。

```
SELECT Cno, Ccredit  
FROM Course  
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';  
WHERE Cname LIKE "DB/_Design" ESCAPE "/";
```

[例20] 查询以"DB_"开头，且倒数第3个字符为 i 的课程的具体情况。

```
SELECT *  
FROM Course  
WHERE Cname LIKE 'DB\__%i\__' ESCAPE '\';
```

ESCAPE '\' 表示 “ \ ” 为换码字符



实例-练习

82

```
SELECT Cno FROM Course WHERE Cname LIKE "DB\_Design";
```

```
SELECT Cno FROM Course WHERE Cname LIKE "DB\_Design";
```

```
SELECT Cno FROM Course WHERE Cname LIKE "DB/_Design" escape "/";
```

结果分别是什么？

Cno	Cname	Cpno	Ccredit
1	DB_Design	1	NULL
2	DBEDesign	2	NULL
NULL	NULL	NULL	NULL

#right 能找到DB_Design

#right,这里_表示通配符, 它能找出来DB_Design 和 DBEDesign

#right,这里_不表示通配符, 它能找出来DB_Design



(5) 涉及空值的查询

83

- 谓词: IS NULL 或 IS NOT NULL
- “IS” 不能用 “=” 代替

[例21] 某些学生选修课程后没有参加考试, 所以有选课记录, 但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

```
SELECT Sno, Cno  
FROM SC  
WHERE Grade IS NULL
```

[例22] 查所有有成绩的学生学号和课程号。

```
SELECT Sno, Cno  
FROM SC  
WHERE Grade IS NOT NULL;
```



(6) 多重条件查询

84

- 逻辑运算符：AND和 OR来联结多个查询条件
 - AND的优先级高于OR
 - 可以用括号改变优先级
- 可用来实现多种其他谓词
 - [NOT] IN
 - [NOT] BETWEEN ... AND ...



多重条件查询（续）

85

[例23] 查询计算机系年龄在20岁以下的学生姓名。

```
SELECT Sname  
FROM Student  
WHERE Sdept= 'CS' AND Sage<20;
```



多重条件查询（续）

86

□ 改写[例12]

[例12] 查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别。

```
SELECT Sname, Ssex  
FROM Student  
WHERE Sdept IN ( 'IS', 'MA', 'CS' )
```

可改写为：

```
SELECT Sname, Ssex  
FROM Student  
WHERE Sdept= ' IS ' OR Sdept= ' MA' OR Sdept= ' CS ';
```



练习

87

- 在Select字句中，关键字（）用于消除重复项
 - A. AS B. DISTINCT C. TOP D. PERCENT

- 要用模糊查询从数据库中查找与某一数据相关的所有元组，可以用（）关键字。
 - A. AND B. OR C. ALL D. Like

- 在SQL中，下列涉及空值的操作，不正确的是（ ）。
 - A. AGE IS NULL B. AGE IS NOT NULL
 - C. AGE = NULL D. NOT (AGE IS NULL)



练习

88

- 设有如下关系表R、S和T
 - R(BH, XM, XB, DWH)
 - S(DWH, DWM)
 - T(BH, XM, XB, DWH)
- 写出实现下列关系代数的SQL语句:

1) $\sigma_{DWH='100'}(R)$

2) $\Pi_{XM, XB}(R)$

3) $\Pi_{XM, DWH}(\sigma_{XB='女'}(R))$



3.4.1 单表查询

89

- 查询仅涉及一个表：
 - 一、 选择表中的若干列
 - 二、 选择表中的若干元组
 - 三、 **ORDER BY**子句
 - 四、 聚集函数
 - 五、 **GROUP BY**子句



三、ORDER BY子句

90

- **ORDER BY子句（对结果进行排序）**
 - 可以按一个或多个属性列排序
 - 升序：**ASC**；降序：**DESC**；缺省值为升序
- 当排序列含空值时
 - **ASC**：排序列为空值的元组最后显示
 - **DESC**：排序列为空值的元组最先显示



ORDER BY子句（续）

91

[例24] 查询选修了3号课程的学生学号及其成绩，查询结果按分数降序排列。

```
SELECT Sno, Grade  
FROM SC  
WHERE Cno= ' 3 '  
ORDER BY Grade DESC;
```

[例25] 查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列。

```
SELECT *  
FROM Student  
ORDER BY Sdept, Sage DESC;
```



3.4.1 单表查询

92

- 查询仅涉及一个表：
 - 一、 选择表中的若干列
 - 二、 选择表中的若干元组
 - 三、 **ORDER BY**子句
 - 四、 聚集函数
 - 五、 **GROUP BY**子句



四、聚集函数

93

□ 聚集函数:

□ 计数

COUNT ([DISTINCT|ALL] *)

COUNT ([DISTINCT|ALL] <列名>)

□ 计算总和

SUM ([DISTINCT|ALL] <列名>)

□ 计算平均值

AVG ([DISTINCT|ALL] <列名>)

□ 最大最小值

MAX ([DISTINCT|ALL] <列名>)

MIN ([DISTINCT|ALL] <列名>)

An Introduction to Database System

3/31/2018



聚集函数（续）

94

[例26] 查询学生总人数。

```
SELECT COUNT(*)  
FROM Student;
```

[例27] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT Sno)  
FROM SC;
```

[例28] 计算1号课程的学生平均成绩。

```
SELECT AVG(Grade)  
FROM SC  
WHERE Cno= '1';
```



聚集函数（续）

95

[例29] 查询选修1号课程的学生最高分数。

```
SELECT MAX(Grade)
FROM SC
WHERE Cno= '1' ;
```

[例30] 查询学生200215012选修课程的总学分数。

```
SELECT SUM(Ccredit)
FROM SC, Course
WHERE Sno='200215012' AND
SC.Cno=Course.Cno;
```



3.4.1 单表查询

96

- 查询仅涉及一个表：
 - 一、 选择表中的若干列
 - 二、 选择表中的若干元组
 - 三、 **ORDER BY**子句
 - 四、 聚集函数
 - 五、 **GROUP BY**子句



五、GROUP BY子句

97

□ GROUP BY子句分组：

细化聚集函数的作用对象

- 未对查询结果分组，聚集函数将作用于整个查询结果
- 对查询结果分组后，聚集函数将分别作用于每个组
- 作用对象是查询的中间结果表
- 按指定的一列或多列值分组，值相等的为一组



GROUP BY子句（续）

98

【例31】 求各个课程号及相应的选课人数。

```
SELECT Cno, COUNT(Sno)
FROM SC
GROUP BY Cno;
```

查询结果：

Cno	COUNT(Sno)
1	22
2	34
3	44
4	33
5	48



GROUP BY子句 (续)

99

[例32] 查询选修了3门以上课程的学生学号。

```
SELECT Sno  
FROM SC  
GROUP BY Sno  
HAVING COUNT(*) >3;
```



GROUP BY子句（续）

100

- **HAVING**短语与**WHERE**子句的区别：
 - 作用对象不同
 - **WHERE**子句作用于基表或视图，从中选择满足条件的元组
 - **HAVING**短语作用于组，从中选择满足条件的组。



练习

101

- 若SQL语句中的Order By短语指定了多个字段，则（ ）
 - A.无法排序 B.只按第一个字段排序
 - C.按自左至右的字段顺序排序 D.按自右至左的顺序排序
- 下列哪个函数不属于聚集函数（ ）。
 - A、count() B、avg() C、min() D、str()
- 聚集函数的返回值是（ ）。
 - A、一个标量值 B、一组值 C、表达式 D、表
- 在SQL查询语句中,GROUP BY语句用于(), 如果要求分组满足指定条件，则需要使用（ ）子句来限定分组。
 - A.选择行条件 B.对查询进行排序 C.列表 D.分组查询
 - E. Ordering F. Having G. Giving H. Holding



练习

102

- 下列各运算符中 () 不属于逻辑运算符。
 - A、& B、not C、and D、or
- 下列哪条语句能够从学生表中查询出姓名的第二个字是“敏”的学生的信息 () 。
 - A、select * from 学生表 where 姓名=' _敏%'
 - B、select * from 学生表 where 姓名 like ' _敏%'
 - C、select * from 学生表 where 姓名 like ' %敏%'
 - D、select * from 学生表 where 姓名 like ' %敏'
- 查询家庭地址与“上海”有关的记录应该用()。
 - SELECT * FROM 家庭信息 WHERE 家庭地址 LIKE '*上海*'
 - B. SELECT * FROM 家庭信息 WHERE 家庭地址= '%上海%'
 - C. SELECT * FROM 家庭信息 WHERE 家庭地址 LIKE '?上海?'
 - D. SELECT * FROM 家庭信息 WHERE 家庭地址 LIKE '%上海%'



下课了。。。

